GlucoEdge: Calibration Coverage and Quantization Trade-offs in On-Device Glucose Trend Classification

Mohamed Menasy
Independent Researcher
mohamedmenasy@gmail.com

October 2026

Abstract

How does calibration-data coverage affect minority-class performance and deployment cost in a compact, quantized glucose trend classifier? We answer this with a controlled study on GlucoEdge, an open-source Android and iOS prototype that maps 12 five-minute continuous glucose monitor (CGM) readings to one of five rate-of-change classes 15 minutes ahead. On a corrected evaluation protocol — split-role provenance masks, gap-free windows, and a separate held-out-participant component — we trained a 2,909-parameter CNN at five seeds on public Weinstock data prepared by GlucoBench, then quantized each checkpoint with four calibration-selection strategies at two budgets and three draws: 100 INT8 artifacts, evaluated pairwise against their float sources. The toolchain-default sequential selection reproduces the failure we previously shipped: the first eligible windows cover two participants, cap the INT8 input range at 251 mg/dL, and cost 13.4 percentage points of held-out directional recall (95% CI 10.3–16.9) relative to a uniform random selection of the same size, while raw accuracy rises — an illusion created by the majority class. Any randomized 200-window selection restores float-level directional recall (0.487 vs. 0.487); range-aware stratification adds nothing further (+0.06 points, CI −0.02 to +0.13), a five-fold calibration budget changes it by at most +0.08 points, and a min–max observer behaves like the default histogram observer. Clip-only controls attribute the entire sequential-arm loss to input-range clipping rather than grid resolution, and every INT8 artifact is byte-identical in size, so the fix costs nothing at deployment time: calibration coverage here means covering participants, not tuning budgets or samplers. The CNN outperforms recent-slope and logistic-regression baselines on directional recall. Prior single-device latency observations are reported as historical context; the new artifacts have not been device-timed. GlucoEdge is a research demonstration, not a medical device or treatment system.

1 Introduction

A CGM sampled every five minutes produces 12 observations per hour. These readings can be used to predict a near-future glucose value, event, or trend [1]. Here, we study such predictions for research purposes; they are not intended to guide treatment.

These targets define different learning problems. Value regression estimates a future glucose level, event prediction asks whether a glycemic event will occur, and trend classification assigns a directional rate category. Published studies also differ in prediction horizon, use of contextual inputs, cohort, split construction, and metric. Reviews and comparison indices document these differences. Public benchmarks and cross-dataset experiments help distinguish performance within a dataset from transfer to another data source [1–4]. Scores from incompatible protocols should therefore not be compared directly.

Commercial CGM trend arrows are related but not equivalent targets. They summarize recent or current glucose direction and rate using device-specific definitions [5, 6]. In contrast, GlucoEdge derives each label from the change between the current sample and the sample 15 minutes later. The arrow-like rate bands provide a compact research representation, but the future-derived label neither reproduces a manufacturer's algorithm nor provides a dosing rule.

Deployment raises further questions. Conversion may change a model's outputs, and the client may need different input handling. Lower precision also does not guarantee lower latency on a given device. Class imbalance can hide conversion errors: accuracy can increase as predictions concentrate on a majority class even while minority-class recall declines [7].

The research question is: *how does calibration-data coverage affect minority-class performance and deployment cost in compact glucose trend classifiers?* Here, coverage refers to the input values and trajectories represented during quantizer calibration, not calibration of predicted probabilities. An earlier revision of this work could only motivate the question: a provenance audit traced the shipped INT8 artifact's input range to a single participant's 17.5 hours of data, whose maximum set

a ceiling below readings found in a quarter of training windows, but no comparison of calibration selections existed. This revision runs that comparison under a prespecified protocol, frozen — hypotheses, samplers, endpoint, missing-class rules, and environment — before any held-out label was read.

The study holds the architecture, loss, quantizer, and data split fixed and varies only calibration selection: four strategies (the toolchain-default sequential order, uniform random, participant-balanced random, and participant-balanced range coverage), two budgets (200 and 1,000 windows, nested), three random draws, and five training seeds, giving 100 INT8 artifacts evaluated against their five float sources, plus a ten-artifact min–max-observer arm. The evaluation protocol is corrected relative to the earlier revision: split-role provenance masks remove context-only targets, a gap-fill mask removes windows touched by interpolation of missing readings, and held-out-participant evaluation is separated from temporal evaluation on training participants. This paper reports:

- a corrected, provenance-masked evaluation protocol for GlucoBench-formatted CGM windows, with eligible-window and class-support audits published alongside the code;
- a controlled answer to the coverage question: randomizing a 200-window calibration selection recovers all of the directional recall the default sequential selection loses (13.4 points, 95% CI 10.3–16.9), while range-aware stratification, a five-fold budget, and an observer change add nothing measurable;
- mechanism controls that attribute the sequential-arm loss to input-range clipping rather than input-grid resolution or internal quantization; and
- deployment-cost accounting: every INT8 artifact has identical size, sampler and export times are fractions of a second, so the corrected calibration is free at deployment time. Prior single-device latency observations are retained as historical context.

All source code, tests, synthetic replay data, and shipped model artifacts are available in the project repository.[1]

2 Related Work

2.1 Forecasting tasks, horizons, and inputs

CGM forecasting has expanded from autoregressive value prediction using sensor history [8] to recurrent and attention models trained with CGM and contextual variables [9]. Recurrent models have also been used to estimate predictive variance rather than only a point value [10], while convolutional–recurrent models fuse CGM, insulin, and meal sequences before predicting 30- or 60-minute values [11]. Architecture and input modality change together across these studies, so their results do not isolate the effect of model family alone. Cross-dataset evaluation asks a different question: whether a trained model generalizes when the source dataset, cohort, and collection process change [4].

Table 1 summarizes these design differences. It distinguishes offline predictive evaluation from reported device execution and omits scores that would be misleading to compare across protocols.

2.2 Benchmarks and evaluation protocols

The split determines what a study can claim about generalization. Subject-dependent splits evaluate people represented during training; held-out-participant splits evaluate unseen people within the same dataset. Cross-dataset evaluation also changes the cohort and data pipeline. GlucoBench standardizes preprocessing, splits, and baselines across five public datasets [1]; experiments across four datasets directly test changes in data source [4].

GlucoEdge uses GlucoBench's largest type-1 diabetes cohort, referred to as Weinstock. Its source study collected blinded CGM from older adults with long-standing type-1 diabetes [12].

Time-series splitting also requires care because nearby and overlapping windows are dependent. The validity of cross-validation for autoregressive prediction depends on the residual structure rather than a generic random-split rule [13], and leakage can produce answers to a less demanding problem than the one a study claims to test [14]. Even with a fixed split, training and implementation choices contribute benchmark variance [15]. Results therefore need to state the participant, temporal, and dataset boundaries used in evaluation.

2.3 Trend representations and class imbalance

Commercial trend arrows summarize recent or current movement using device-specific categories whose rate bands and look-back intervals differ [5, 6]. A future label instead asks whether the trajectory after the input window falls, remains stable, or rises. GlucoEdge uses these rate bands to define a research target from the sample 15 minutes after the window. It does not reproduce a commercial arrow algorithm or support treatment decisions.

In the Weinstock windows used here, most labels are stable and faster changes are relatively rare. Overall accuracy therefore weights the stable class in proportion to its prevalence. Macro-averaged recall, equivalent to balanced accuracy for this single-label setting, gives each

[1] `https://github.com/mohamedmenasy/GlucoEdge`

Table 1: Representative CGM forecasting studies and evaluation scope. Metrics are not compared across rows because datasets and protocols differ.

Study	Target	Horizon	Inputs	Evaluation scope	Device evidence
Reifman et al. [8]	Value	30 min	CGM	9 participants	None reported
Mirshekarian et al. [9]	Value	30/60 min	CGM + context	OhioT1DM/synthetic	None reported
Martinsson et al. [10]	Value + variance	30/60 min	CGM	OhioT1DM	None reported
Li et al. [11]	Value	30/60 min	CGM, insulin, meals	10 real + 10 simulated	Android timing
GlucoBench [1]	Value trajectory	60 min	Dataset-dependent	Five public datasets	None reported
Ghimire et al. [4]	Value	30/60 min	CGM	Four datasets	None reported
GlucoEdge (this work)	Trend class	15 min	CGM	Weinstock held-out + temporal	Android/iOS replay timing

class equal weight and makes minority-class degradation visible [7]. Macro recall still does not establish event-level or clinical performance.

2.4 Edge inference and quantization

One-dimensional CNNs extract local temporal features with compact feed-forward networks [16].

Mobile inference systems can reduce neural-network execution cost through compression and runtime-specific scheduling [17], but latency is a property of the model, runtime, and hardware together. Li et al. reported Android timing for a glucose forecasting model [11]; broader smartphone and tiny-device benchmarks also measure specified workloads and platforms because model size alone does not determine speed [18, 19].

Integer quantization approximates floating-point tensors with discrete values parameterized by a scale and zero point [20]. Static post-training quantization estimates activation ranges from calibration inputs before conversion [21, 22]. Low-bit studies show that clipping, range estimation, and rounding choices can determine the resulting error [23, 24]; specialized methods have also been proposed to improve post-training accuracy when calibration sets are small [25]. These findings support checking converted outputs and per-class metrics alongside model size.

GlucoEdge connects public preprocessing with conversion parity, class-wise evaluation, and offline replay on one Android and one iOS device; it does not establish live-sensor, clinical, or cross-hardware performance.

3 Problem Formulation

Let g_t be the CGM value at time t, in mg/dL, with observations every five minutes. The input is

$$\mathbf{x}_t = [g_{t-55}, g_{t-50}, \ldots, g_t] \in \mathbb{R}^{12}. \tag{1}$$

The future rate of change is computed from a point outside the input window:

$$r_t = \frac{g_{t+15} - g_t}{15} \quad [\text{mg/dL/min}]. \tag{2}$$

The class y_t is assigned by

$$y_t = \begin{cases} \texttt{falling_fast}, & r_t \le -2, \\ \texttt{falling}, & -2 < r_t \le -1, \\ \texttt{stable}, & -1 < r_t < 1, \\ \texttt{rising}, & 1 \le r_t < 2, \\ \texttt{rising_fast}, & r_t \ge 2. \end{cases} \tag{3}$$

The 12 readings span 55 elapsed minutes; "one-hour history" refers to 12 samples at a five-minute cadence. The history length and 15-minute horizon are fixed engineering choices for this prototype. The five-class target is a simplification based on two samples and does not describe the full intervening trajectory. The boundaries use rate bands commonly represented by CGM trend arrows, while collapsing device-specific arrow taxonomies into five classes [5, 6]. No threshold was optimized on the test labels.

4 Data and Methods

4.1 Dataset and preprocessing

The experiments use the Weinstock data distributed through GlucoBench. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12]. GlucoBench reports 200 participants before preprocessing and 192 afterwards, with a five-minute sampling interval [1]; our audit of the formatter output reproduces both counts. The configuration used here disables scaling and retains glucose in raw mg/dL. The formatter rounds timestamps to the minute, splits each participant's record at gaps longer than 45 minutes, drops segments with fewer than 240 original readings, and linearly interpolates each remaining segment onto a five-minute grid that starts at its first reading. Each segment is tagged with an `id_segment`. Of the 579,782 grid values, 75.8% coincide with an original reading. A further 19.8% fall between two consecutive readings no more than 7.5 minutes apart, because reading times drift relative to the grid; 4.4% fill one or more missing readings. Because interpolation precedes windowing, interpolated values may occur among the inputs or at samples used to form targets. The configured split holds out 10% of participants (20 people) for an out-of-distribution component and forms temporal validation

Table 2: Window distribution after GlucoBench formatting. Percentages are within each split.

Class	Train	Aggregate test
`falling_fast`	13,260 (3.12%)	3,692 (2.90%)
`falling`	40,122 (9.45%)	11,387 (8.95%)
`stable`	314,257 (74.04%)	95,998 (75.49%)
`rising`	39,428 (9.29%)	11,012 (8.66%)
`rising_fast`	17,395 (4.10%)	5,076 (3.99%)
Total	424,462	127,165

Table 3: Eligible windows under the corrected protocol. Scored removes context-only and boundary-crossing targets; eligible additionally removes windows touched by gap filling.

Component	Frame	Scored	Eligible	Part.
Training	424,462	424,462	328,575	172
Calibration pool	55,289	35,560	27,109	127
Temporal test	127,165	52,540	37,361	166
Held-out test		57,068	46,376	20

and test components from the remaining participants. The formatter combines the test components into one frame; Section 4.2 separates them again by row ownership, and all results report the held-out-participant and temporal components separately.

Windows are generated independently within each (`id`, `id_segment`) group and ordered by time. A window is emitted only when all 12 input points and the three following horizon steps fit in the same segment. The stride is one reading, matching an application that updates every five minutes. This rule prevents a sensor gap or participant boundary from being treated as a continuous trajectory. Adjacent stride-one windows overlap heavily and are statistically dependent. The resulting frames yielded 424,462 training, 55,289 validation, and 127,165 aggregate test windows from 172, 127, and 186 participants, respectively. The test windows comprise 57,068 from the 20 held-out participants and 70,097 from temporal segments of 166 training participants.

Table 2 shows the resulting class imbalance. Both fast classes exceed 13,000 training windows. A preliminary 20-epoch run obtained test recalls of 0.5333 and 0.5380 for `falling_fast` and `rising_fast`, respectively. Neither met the predeclared condition for collapsing the classes: fewer than 200 training examples or recall below 0.30. The deployed model therefore retains all five classes. Test recall informed this five-versus-three decision, so it must be treated as an exploratory development choice rather than independent validation.

4.2 Split roles, observation mask, and eligible windows

GlucoBench repeats rows from a preceding split at the top of validation and in-distribution test frames so sequence models can form initial histories. The earlier revision of this work let every prefix row start a window, so 13.6% of aggregate test windows scored targets in validation-owned rows and 35.1% of validation windows duplicated training windows (Table 4). The corrected protocol assigns every formatter row to the earliest split that lists it and scores a window in a split only when its anchor (last input sample) and all three forecast rows are owned by that split. Earlier-split readings may serve as causal history among the inputs. Scored (participant, segment, target time) keys are asserted disjoint across splits.

Because the formatter interpolates before windowing, a second mask removes gap filling: a grid value is retained only if it lies on or between two original readings at most 7.5 minutes apart. An audit showed that requiring exact coincidence with an original reading would discard roughly 40% of windows for grid drift unrelated to missing data, so the gap-fill rule is the primary mask and exact coincidence is a sensitivity tier. A window is eligible when its 12 inputs and future endpoint are all gap-free; the two intermediate forecast steps are not label inputs and are not masked. Table 3 shows the result. Eligible training windows retain all five classes with at least 10,330 examples (`falling_fast`); the calibration pool keeps 127 of 172 development participants.

4.3 Model and training

The `TrendCNN` accepts tensors of shape $(B, 1, 12)$. It applies a one-dimensional convolution from 1 to 24 channels, a second convolution from 24 to 36 channels, ReLU after each layer, adaptive average pooling over time, and a 36-to-5 linear head. Both convolutions use kernel size 3, stride 1, and padding 1. The network contains 2,909 trainable parameters.

Training uses inverse-frequency class-weighted cross-entropy, batch size 64, and Adam with learning rate 10^{-3} for 20 epochs [26, 27]. The study trains five checkpoints with seeds 0–4 on the eligible windows of Section 4.2; the fixed split does not change with the training seed. Validation loss was observed after every epoch; there was no early stopping or test-time selection. Classification metrics were computed with scikit-learn and the study's own confusion-count code [28]. Because the test distribution is dominated by `stable`, the primary summary is the unweighted mean recall of the four directional classes (minority macro recall); five-class macro recall, equivalent to balanced accuracy, is also reported [7].

Two input-dependent baselines use the same eligible windows. The recent-slope baseline maps $(g_t - g_{t-15})/15$, computed from the inputs alone, through the same five thresholds. The logistic-regression baseline is a multinomial model on the 12 readings with training-only standardization, balanced class weights, and fixed $C = 1$; it

converged. The constant-`stable` classifier shows the effect of prevalence. The shipped application artifact (seed 0, historical protocol) is unchanged; no new artifact replaces it in the applications.

5 Conversion and Deployment

5.1 Shipped artifacts (historical protocol)

The shipped application checkpoint was converted with LiteRT Torch. Float conversion uses a fixed $(1, 1, 12)$ input signature. INT8 conversion uses PyTorch 2 Export (PT2E) with LiteRT Torch's `get_symmetric_quantization_config` helper (`is_per_channel=True`, `is_dynamic=False`). Despite the helper's name, only the weights are quantized symmetrically, per channel. Activations, including the model input, use per-tensor affine quantization whose ranges a histogram observer selects to minimize quantization error [21]. The first 200 validation windows serve as calibration inputs. Selection follows the dataset's participant/segment and time order, with no randomization or attempt to balance input-range coverage; Section 7.1 shows that all 200 windows come from one participant. The converted model's input scale is 0.9408 mg/dL per code with zero point -128, so its representable input range is 0–239.9 mg/dL. The range starts at zero because the observer always includes zero, which leaves the codes below the lowest CGM reading unused.

5.2 Calibration experiment design

The study freezes the checkpoint, quantizer, and conversion pipeline and varies only which validation windows calibrate the histogram observer. The calibration pool is the 27,109 eligible validation windows; no sampler reads future labels, and selection code is tested to be invariant under label shuffling. Four strategies are compared. *Sequential* takes the first windows in deterministic (participant, segment, time) order — the toolchain-default pattern that produced the shipped artifact. *Uniform random* is a seeded permutation of the pool. *Participant-balanced random* fixes a shuffled participant order per draw and cycles it round-robin, taking one window from a fixed per-participant permutation each visit and skipping exhausted participants. *Participant-balanced range coverage* reuses exactly the same participant allocation but, within each participant, cycles through low/middle/high range cells defined by the pool's own 10th and 90th percentiles of per-window input minimum and maximum, so it differs from the balanced-random arm only in within-participant input-range coverage.

Each strategy is drawn at budgets of 200 and 1,000 windows, with the 200-window set nested as the first 200 of the 1,000-window ordering. Stochastic strategies use three seeded draws (101–103); all selections are shared across the five model seeds. This yields 100 INT8 artifacts (5 seeds $\times$ [2 sequential + 3 strategies $\times$ 2 budgets $\times$ 3 draws]). A mechanism-control arm re-quantizes two cells (sequential-200 and range-coverage-200, draw 101) with a min–max activation observer in place of the histogram observer, adding 10 artifacts. For every calibration set the study records participants, segments, unique input readings, input extremes and quantiles, and occupied range cells, because a nominal window count can cover few distinct readings; for every artifact it records input quantization parameters, size, and SHA-256.

5.3 Client implementations

For input scale s and zero point z, both clients use the affine form

$$q = \mathrm{clip}\left(\mathrm{round}(x/s) + z, -128, 127\right), \qquad \hat{x} = s(q - z). \tag{4}$$

Quantization parameters are read from the model metadata rather than hardcoded. All three clients divide in single precision and round exact half-way values to the even integer: Python with `numpy.round`, Kotlin with `kotlin.math.round`, and Swift with `rounded(.toNearestOrEven)` [29–31]. They therefore produce identical integer inputs for any glucose value. The July device tests ran an earlier Kotlin version that rounded ties toward positive infinity. The golden-vector inputs are given to 0.1 mg/dL, and none of those values is a tie at the shipped input scale, so the expected outputs are unchanged.

The Kotlin/Jetpack Compose application replays a bundled synthetic trace. A 12-reading ring buffer resets after a cadence gap, mirroring training-time segment boundaries. Every complete window is classified locally; the merged Android manifest is tested to ensure it does not request the `INTERNET` permission. The float artifact is the default and INT8 can be selected for comparison.

Real devices use LiteRT 2.1.0 `CompiledModel` on CPU [32]. On the tested Apple-Silicon-hosted Android emulator, native model creation terminated with an illegal-instruction fault during a CPU feature probe for the Arm Scalable Matrix Extension (SME). Because this fault cannot be caught by Kotlin, emulators are selected before runtime initialization and use the classic interpreter with XNNPACK disabled. This workaround applies to the tested environment; it does not establish performance across LiteRT configurations.

A SwiftUI iOS application mirrors the Android client. It bundles the same two artifact files and synthetic trace by reference to the Android asset directory, so both platforms ship byte-identical models. It runs them through LiteRT's Swift `CompiledModel` API on CPU, using the LiteRT 2.3.0 release binaries. Replay, the gap-resetting window buffer, the model switch, and the latency statistics follow the Kotlin implementation. iOS has no equivalent of Android's `INTERNET` permission. Instead, a source

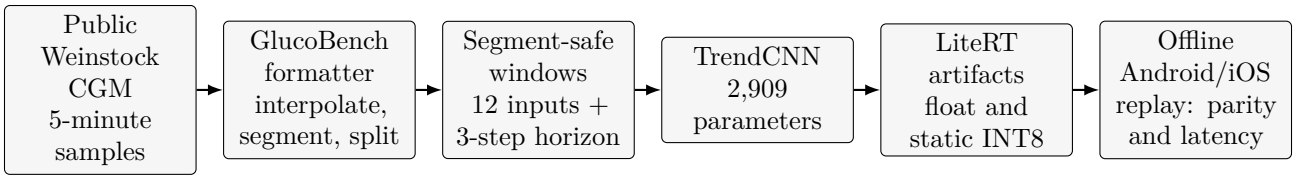


Figure 1: End-to-end GlucoEdge workflow. Patient data are used locally for training and evaluation; only model files and a synthetic replay trace are committed to the Android and iOS applications.

scan fails the build's continuous-integration job if networking APIs appear in the application code. This guard is weaker than the Android manifest check.

6 Experimental Protocol

Evaluation uses two disjoint components from Section 4.2: the 46,376 eligible windows of the 20 held-out participants (primary) and the 37,361 eligible temporal windows of 166 training participants (secondary). They are never concatenated. The protocol — hypotheses, samplers, budgets, endpoint, bootstrap, and missing-class rules — was frozen and committed before any held-out label was read, in `paper/results/calibration-coverage/protocol.json`. Because an earlier revision of this work evaluated on an overlapping aggregate frame and the five-class decision was informed by a historical test run, the study is a locked, corrected reanalysis of previously touched data, not a fresh test set.

A provenance audit (`experiments/split_audit.py`) maps every window to the formatter rows it uses and assigns each row to the earliest split that lists it. It reads row identities and timestamps for all splits but glucose values only from training and validation rows. It also records the composition of the shipped calibration set and the shipped INT8 model's input quantization parameters.

If n_c is class support, $N = \sum_c n_c$, and R_c is class recall, then

$$A = \sum_{c=1}^{5} \frac{n_c}{N} R_c, \qquad R_{\text{macro}} = \frac{1}{5} \sum_{c=1}^{5} R_c, \tag{5}$$

and the primary endpoint is the unweighted mean recall of the four directional classes. The prespecified primary contrast is range-coverage versus participant-balanced random at budget 200, which isolates within-participant input-range coverage from participant representation. Secondary contrasts are range coverage versus uniform random, 1,000 versus 200 within each strategy, and uniform random versus sequential at each budget. A recall is undefined on zero support and never replaced by zero.

Uncertainty comes from a paired participant-cluster bootstrap: 2,000 draws with a fixed analysis seed resample participants with replacement, keep every selected participant's complete windows, recompute each arm's confusion counts, and average the paired differences over the five model seeds and three calibration draws inside each bootstrap draw. Percentile 95% intervals are conditional on the fitted models and calibration sets. Overlapping windows are never resampled individually. If more than 5% of draws have an undefined endpoint, the interval is omitted.

Two input-grid controls separate mechanisms. The float-QDQ control feeds each INT8 artifact's quantize–dequantize image of the inputs — using that artifact's own scale and zero point — through the float model, which measures the input-grid intervention alone. The clip-only control clips inputs to the artifact's representable range without re-gridding, which separates range truncation from input resolution. Clipping is counted with the actual quantization operation (round half-to-even, then saturate), per input slot and per window, with true-class breakdowns on the evaluation components only. These comparisons describe the measured error components; they are not an additive causal decomposition of every error.

File size is the exact serialized artifact size. The device observations below predate this study and describe the shipped artifacts only; none of the 110 new artifacts has been device-timed. Device latency is measured inside the application's `classify` method with `System.nanoTime`; it includes buffer writes, input quantization when needed, model execution, and output reading/dequantization. The timer stops before softmax, class selection, and construction of the returned prediction; model loading and UI updates are also outside its scope. Measurements were recorded during replay on a Samsung Galaxy S22 Ultra (SM-S908E, Android 16). The float statistics use the latest 100 inferences, while the recorded INT8 statistics contain 21 inferences after a model switch. The application reports p95 as the sorted sample at zero-based index $\lfloor 0.95(n-1) \rfloor$, so these estimates also have different rank resolution. The timings cover this portion of the inference path during replay. They measure neither the full classification call nor the kernel alone, and the runs were not controlled for a benchmark comparison.

The iOS client times the same region with the monotonic `DispatchTime` clock. Measurements were recorded on an iPhone 18 Pro Max (iPhone19,3, iOS 27.0.1) during 16× replay of the synthetic trace, which produces one

inference every 312.5 ms. Each model's statistics use its latest 100 inferences, and the measurement was repeated in two rounds. The application code was a debug build; LiteRT itself is a prebuilt release binary. A separate test-runner benchmark calls each model 100 times back to back after one warm-up call, in three runs. It isolates the timed region from the interval between replay ticks. Calibration preparation time, peak memory, model-load time, and energy were not measured.

Cross-runtime parity was tested with 20 synthetic golden windows, four selected from each float-predicted class and including values above the INT8 input range. The test also verifies the SHA-256 hash of each bundled artifact. Float logits must match the Python output within 10^{-5}; dequantized INT8 outputs are required to match bit-for-bit because both sides dequantize identical integer outputs with the same float32 operation. The iOS suite applies the same vectors and tolerances and also checks the class-name order. It runs on the physical iPhone, on the iOS simulator, and in continuous integration.

7 Results

7.1 Split and calibration provenance

Table 4 summarizes the audit. No test window has a target in a training row, and none repeats a training window. The context-prefix overlap instead affects 17,266 test windows (13.6%) whose targets fall in rows that GlucoBench assigns to validation; validation labels were not used for fitting or checkpoint selection. Most validation frames begin with 192 training rows of model context, so 19,402 validation windows (35.1%) repeat training windows exactly.

The calibration set is narrower than its size suggests. Its 200 overlapping windows come from one participant and one segment: 211 distinct readings spanning 17.5 hours, with 178 windows repeating training windows. The largest calibration input, 240.0 mg/dL, matches the upper end of the INT8 input range, 239.9 mg/dL. In development data, 24.4% of training windows and 26.3% of validation windows contain at least one reading above that range. Figure 2 shows the resulting ceiling against the development inputs. These historical facts motivated the controlled study; the remaining subsections report its results.

7.2 Coverage manipulation

Table 5 shows that the sampling interventions changed measured coverage, not just the strategy label. The sequential-200 set draws from 2 of 127 pool participants and 224 unique readings with a maximum of 251 mg/dL; every randomized 200-window set covers 96–127 participants, roughly 2,300 unique readings, and the pool maximum of 401 mg/dL. The resulting INT8 input ranges

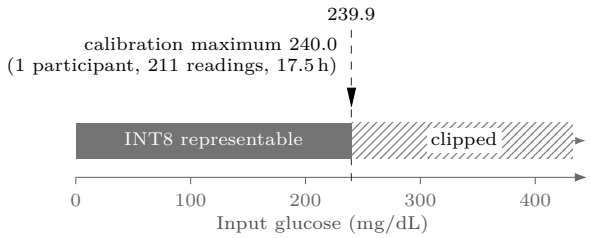


Figure 2: The INT8 input range ends at 239.9 mg/dL, the quantized image of the calibration-set maximum. Above that ceiling lie 18.5% of training readings, and 24.4% of training windows contain at least one such reading (validation: 20.5% and 26.3%). Inputs there are clipped to 239.9 mg/dL.

Table 4: Window provenance and calibration-set composition from the audit. Shares are within each split.

Window source	Windows	Share
Validation, target in validation rows	35,887	64.9%
Validation, copy of a training window	19,402	35.1%
Test, held-out participants	57,068	44.9%
Test, target in test rows	52,831	41.5%
Test, target in validation rows	17,266	13.6%
Test, target in training rows	0	0.0%
INT8 calibration set (first 200 validation windows)		
Participants; segments		1; 1
Distinct input readings; span		211; 17.5 h
Copies of training windows		178 of 200
Maximum input		240.0 mg/dL
INT8 input range		0–239.9 mg/dL

split accordingly: 0–250.9 (scale 0.98 mg/dL per code) for sequential-200 against 0–400.8 (scale 1.57) for the randomized arms — the price of the wider range is a 60% coarser input grid. Sequential at budget 1,000 reaches 6 participants, 1,180 readings, and the full range.

7.3 Calibration effects on directional recall

Table 6 and Figure 3 give the main result on the held-out-participant component. Float LiteRT is prediction-identical to its PyTorch checkpoint at every seed. Quantized with the sequential-200 calibration, mean directional recall drops from 0.487 to 0.354 while accuracy *rises* from 0.507 to 0.570 — the same accuracy illusion the shipped artifact showed, now reproduced under the corrected protocol. Quantized with any randomized 200-window selection, INT8 matches float to the third decimal (0.487). `falling_fast` is the most sensitive class: 0.550 float, 0.236 under sequential-200, 0.547 under uniform-200.

Table 7 reports the prespecified contrasts. The primary contrast — range coverage versus participant-

Table 5: Measured coverage of the 200-window calibration sets (draw 101) and the resulting INT8 input range. The pool holds 27,109 eligible windows from 127 participants, range 39–401 mg/dL.

Strategy (200)	Part.	Readings	Max	INT8 range
Sequential	2	224	251	0–250.9
Uniform random	96	2,300	401	0–400.8
Balanced random	127	2,359	401	0–400.8
Range coverage	127	2,358	401	0–400.8

Table 6: Held-out-participant metrics, averaged over five training seeds (and three draws for stochastic strategies). Budget 1,000 in parentheses.

Model	Accuracy	Macro	Directional
Float LiteRT (= PyTorch)	0.507	0.494	0.487
INT8, sequential	0.570 (0.499)	0.412 (0.492)	0.354 (0.487)
INT8, uniform random	0.496 (0.497)	0.492 (0.492)	0.487 (0.488)
INT8, balanced random	0.497 (0.497)	0.491 (0.492)	0.487 (0.488)
INT8, range coverage	0.497 (0.496)	0.492 (0.492)	0.487 (0.488)

balanced random at 200 — is +0.06 percentage points (95% CI −0.02 to +0.13): once participant allocation is randomized, deliberately spreading selections across input-range cells adds nothing measurable. Uniform random versus sequential at 200 is +13.38 points (CI +10.29 to +16.88). Increasing any randomized strategy's budget from 200 to 1,000 changes recall by at most +0.08 points, and sequential itself recovers fully at 1,000, where its window span finally reaches the full input range. The min–max observer arm is indistinguishable from the histogram observer given the same windows (0.354 vs. 0.354 sequential; 0.488 vs. 0.487 range coverage). The temporal component mirrors every pattern with a +12.43-point sequential effect.

7.4 Mechanism: clipping, not resolution

Table 8 decomposes the sequential-200 loss with the input-grid controls. Feeding the float model the clip-only image of the inputs reproduces essentially the entire 13.35-point float-to-INT8 difference (float−clip 13.14; float−QDQ 13.12), so the coarser input grid itself costs nothing measurable and the remaining INT8-internal difference is 0.2 points. Under the sequential-200 range, 27.8% of held-out windows have at least one clipped input slot by the rounded-code rule — 60.9% of `falling_fast` windows, the class that collapses — against none under the randomized range: 3.5% of held-out windows exceed its real-valued top of 400.8 mg/dL at the 401 sensor cap, but those readings round into the top code, so no input slot clips. Wider range coverage therefore did not trade recall for resolution in this model: the grid-resolution cost of covering 39–401 mg/dL was unmeasurable.

Table 7: Prespecified contrasts in mean directional recall (percentage points), paired participant-cluster bootstrap, 2,000 draws. No draw had an undefined endpoint.

Contrast	Held-out	Temporal
Primary: range vs. balanced, 200	+0.06 [−0.02, +0.13]	+0.04 [−0.04, +0.13]
Range vs. uniform, 200	+0.01 [−0.05, +0.07]	+0.02 [−0.07, +0.10]
Uniform vs. sequential, 200	+13.38 [+10.29, +16.88]	+12.43 [+10.28, +14.43]
Uniform vs. sequential, 1,000	+0.05 [−0.05, +0.15]	+0.00 [−0.12, +0.12]
Uniform: 1,000 vs. 200	+0.02 [−0.07, +0.10]	−0.03 [−0.12, +0.06]
Balanced: 1,000 vs. 200	+0.08 [+0.02, +0.15]	+0.04 [−0.05, +0.12]
Range: 1,000 vs. 200	+0.03 [−0.09, +0.16]	−0.01 [−0.11, +0.09]

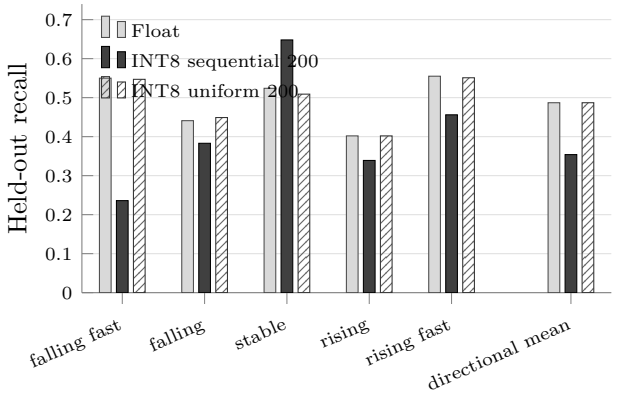


Figure 3: Per-class held-out recall, mean over five seeds (and three draws for the uniform arm). The sequential-200 calibration collapses `falling_fast` and inflates `stable`; a uniform random selection of the same size restores the float profile.

7.5 Baselines

On held-out participants the five-seed float CNN reaches 0.487 directional recall against 0.279 for the recent-slope baseline and 0.461 for logistic regression on the same 12 readings (Table 9). The constant-`stable` classifier has the highest accuracy of all (0.727) and zero directional recall, which is why accuracy cannot rank these models.

7.6 Sensitivity tiers

The protocol froze three disjoint window groups outside or inside the primary mask: the exact-observed subset (every checked row coincides with an original reading), windows with gap-filled inputs but gap-free endpoints, and windows with a gap-filled endpoint (whose labels derive from gap-filled grid values and describe the formatter's grid rather than sensor truth). Table 10 evaluates the same frozen artifacts on each tier. The two findings that matter are mask-insensitive: uniform random calibration beats sequential by +10.5 to +13.2 points of directional recall in every tier and both components (temporal intervals $[+6.5, +14.7]$ at the widest), and the float-to-INT8 gap under uniform calibration never exceeds 1.0 point (largest: +1.0 $[+0.2, +1.9]$ on the temporal gap-filled-endpoint tier). The tier evaluation ran after the primary analysis was published, on

Table 8: Mechanism controls on the held-out component (directional-recall differences in percentage points, seed means). QDQ = float model on the artifact's quantize–dequantize input grid; clip = float model on inputs clipped to the artifact's range without re-gridding. Stochastic arms average all three draws; sequential has one order.

Calibration arm	Float−clip	Float−QDQ	QDQ−INT8
Sequential 200	+13.14	+13.12	+0.23
Uniform 200	+0.01	−0.20	+0.17
Range coverage 200	+0.01	−0.20	+0.16

Table 9: Held-out baselines. The CNN row is the five-seed float mean; baselines are deterministic or single fits.

Model	Accuracy	Macro	Directional
Constant `stable`	0.727	0.200	0.000
Recent slope	0.662	0.384	0.279
Logistic regression	0.405	0.451	0.461
TrendCNN (float)	0.507	0.494	0.487

tiers, artifacts, and contrasts that were all frozen in the protocol beforehand; the full per-model record is in `sensitivity_tiers_r1.json`.

7.7 Second-cohort confirmation

Two further GlucoBench cohorts, untouched by every preceding analysis, received a reduced prespecified matrix (five seeds; sequential versus uniform random at budget 200; per-cohort protocols frozen before any of that cohort's evaluation labels were read). The predictions were paired: a positive uniform−sequential difference on dubosson (type 1 diabetes, nine participants, readings to 400 mg/dL) and a near-zero difference on hall (57 mostly non-diabetic participants, maximum 303 mg/dL). Dubosson's held-out split is a single participant, so its frozen protocol assigns the contrast to the temporal component (six clusters) and reports held-out descriptively; hall keeps held-out (six participants) primary.

Both predictions held, and the magnitudes line up with the mechanism (Table 11): the effect tracks the gap between the sequential calibration ceiling and the population range. Weinstock's ceiling of 239.9 against a 401 maximum cost 13.4 points; dubosson's 286 against 400 costs +2.0 $[-0.0, +3.1]$; hall's 165 against 303 costs +1.3 $[-0.1, +3.3]$ on the primary component — near zero as predicted, though hall's better-powered temporal secondary (50 participants) resolves a small real effect of +4.3 $[+1.1, +7.3]$, consistent with the thin tail of readings above its sequential ceiling. In every cohort the sequential selection again collapsed to two early participants, and uniform calibration again sits within about a point of the float model. Full per-seed records: `confirmation_dubosson.json`, `confirmation_hall.json`.

Table 10: Held-out directional recall by sensitivity tier: five-seed float mean, sequential-200 INT8 mean, uniform-200 INT8 mean (seeds × draws), and the paired cluster-bootstrap uniform−sequential difference.

Tier	n	Float	Seq. 200	Unif. 200	Unif.−Seq. [95% CI]
Exact-observed	36,150	0.476	0.346	0.477	+0.132 [+0.101, +0.166]
Gap-filled inputs	8,682	0.450	0.332	0.446	+0.114 [+0.066, +0.159]
Gap-filled endpoint	2,010	0.501	0.383	0.497	+0.113 [+0.071, +0.152]

Table 11: Second-cohort confirmation (directional recall; five-seed means, uniform over three draws). Δ = uniform−sequential paired cluster bootstrap; dubosson held-out is one participant and gets no interval by its frozen protocol.

Cohort	Component	Float	Seq. 200	Unif. 200	Δ [95% CI]
dubosson	temporal (contrast)	0.489	0.458	0.478	+0.020 [−0.000, +0.031]
dubosson	held-out (1 part.)	0.481	0.472	0.475	+0.003 (descriptive)
hall	held-out (primary)	0.393	0.375	0.388	+0.013 [−0.001, +0.033]
hall	temporal	0.428	0.378	0.421	+0.043 [+0.011, +0.073]

7.8 Delta-encoded inputs

If calibration failures are input-range failures, re-encoding the input should remove them. Two variants were trained from scratch on the same eligible windows and seeds (gates frozen beforehand; like the sensitivity tiers, this is a post-hoc experiment on previously touched held-out data): 11 consecutive differences, and the same differences plus the last absolute value divided by 16, placing a coarse level channel inside the delta range. Both passed the frozen float gate (five-seed held-out directional recall ≥ 0.482): differences alone reach 0.497, and differences plus the scaled anchor reach **0.509** — above the absolute-input baseline's 0.487, so the re-encoding costs nothing and appears to help. The anchor variant proceeded to INT8 per the frozen rule.

The robustness prediction held (Table 12). The anchor model's INT8 input grid is 6.4× finer (scale 0.247 versus 1.572 mg/dL per code), and under the toolchain-default sequential calibration — the selection that costs the absolute-input model 13.4 points — the delta model loses only half a point against its own float (+0.50 $[+0.33, +0.67]$); uniform minus sequential is +0.54 $[+0.38, +0.72]$. The frozen ship criterion (sequential-INT8 within 0.5 points of float) passes at 0.4955, a margin of less than five thousandths of a point, which we state rather than celebrate; the bundled INT8 model is calibrated with the uniform selection, where no gap is resolvable. Full record: `delta_inputs_r1.json`.

7.9 Historical comparison

The earlier revision reported accuracy 0.5184 and macro recall 0.5060 for float and 0.5866/0.4135 for INT8. Those numbers came from a different denominator — the uncorrected aggregate frame, with context-target overlap and no gap mask — and from the shipped sequential calibration, so they are not comparable to Table 6 row

Table 12: Delta-encoded inputs (held-out directional recall, five-seed means; uniform over three draws). The absolute-input row repeats the main study for comparison.

Input encoding	Float	INT8 seq. 200	INT8 unif. 200	Unif.−Seq. [95% CI]
Absolute mg/dL	0.487	0.354	0.487	+0.134 [+0.103, +0.169]
Deltas only	0.497	—	—	(did not proceed)
Deltas + anchor/16	0.509	0.504	0.509	+0.005 [+0.004, +0.007]

by row. The qualitative pattern they showed (accuracy up, every directional recall down) is exactly what the corrected protocol reproduces for the sequential arm.

7.10 Deployment costs, and historical device observations

Calibration coverage has no deployment cost in this model. All 100 matrix INT8 artifacts serialize to exactly 11,976 bytes regardless of strategy or budget (float: 17,352 bytes, a 31.0% reduction), and all 110 INT8 artifacts share a single operator inventory, so no graph change is attributable to any calibration arm. Offline preparation is also negligible: every sampler selects its windows in under 0.03 s, and INT8 export averages 0.5 s for a 200-window set and 0.8 s for a 1,000-window set on the study host. The corrected calibration therefore changes nothing the device sees except the two quantization parameters. The repository's bundled application models were accordingly replaced with the study's seed-0 float artifact and its uniform-200 INT8 artifact; both client golden-parity suites pass against the new files bit-exactly on the continuous-integration simulators.

The prespecified nine-artifact benchmark was then executed on the iPhone 18 Pro Max (iPhone19,3, iOS 27.0.1), the only study device still available: the seed-0 float artifact plus all four strategies at both budgets, 30 paired rounds with the artifact order reshuffled each round (seed 20261010), and per artifact per round 100 warm-up plus 200 timed calls on one frozen synthetic workload — 6,000 timed calls per artifact, with every artifact hash-verified and parity-checked on all 277 workload windows before any timing, and the thermal state nominal in every round (Table 13). The result matches the byte-level argument: the eight calibration arms are indistinguishable (means 3.42–3.47 μs; paired round-level differences against the same-round float mean, +0.64 to +0.70 μs for every arm), so the calibration fix costs nothing measurable on hardware. INT8 remains slightly slower than float back-to-back, consistent with the historical observation that per-call overhead, not arithmetic, dominates; p95 here is linear interpolation over the 6,000 pooled calls. Raw rounds and the aggregation are in `device_benchmark_ios_r1.json`.

Table 14 retains the earlier two-device historical observations for the previously shipped artifacts. In those historical observations neither phone showed an INT8 latency advantage, and the iPhone's back-to-back benchmark (means 2.83–2.85 μs float, 3.69–3.72 μs INT8) indicates that per-call overhead, not arithmetic, dominates the in-app path — the same qualitative picture as the new measurement.

Table 13: Nine-artifact device benchmark (iPhone 18 Pro Max, iOS 27.0.1, back-to-back calls; inner timer: quantize + run + dequantize, no softmax). 6,000 timed calls per artifact over 30 paired rounds. Δ is the mean of paired round-level differences vs. the same round's float mean.

Artifact (seed 0)	Mean	Median	p95	Δ vs. float (μs)
Float	2.78	2.96	3.17	—
Sequential 200	3.44	3.33	4.00	+0.67
Sequential 1000	3.44	3.33	3.96	+0.66
Uniform 200	3.42	3.33	3.96	+0.64
Uniform 1000	3.47	3.38	4.08	+0.70
Balanced 200	3.45	3.33	4.00	+0.67
Balanced 1000	3.44	3.33	3.96	+0.66
Range coverage 200	3.44	3.33	3.96	+0.66
Range coverage 1000	3.45	3.33	4.00	+0.68

Table 14: Historical artifact and physical-device observations for the shipped models (uncorrected protocol era). Timing ends before softmax and class selection. Android sample counts differ by model; iPhone ranges span two rounds of 100 inferences per model.

Measurement	Float	INT8
Artifact size (bytes)	17,352	11,976
Artifact size (KiB)	16.95	11.70
Galaxy S22 Ultra, Android 16, LiteRT 2.1.0		
Mean latency (ms)	0.334 ($n = 100$)	0.484 ($n = 21$)
p95 latency (ms)	0.540	0.665
Golden vectors passed	20/20	20/20
iPhone 18 Pro Max, iOS 27.0.1, LiteRT 2.3.0		
Mean latency (ms)	0.062–0.073	0.101–0.110
p95 latency (ms)	0.103–0.134	0.178–0.182
Golden vectors passed	20/20	20/20

All three Android instrumentation tests passed on the physical device. The `CompiledModel` path loaded both artifact hashes, matched float logits within 10^{-5} on all 20 windows, and matched INT8 dequantized outputs exactly. All four iOS parity tests passed on the iPhone with the same result: the artifact hashes, the class-name order, float logits within 10^{-5}, and bit-exact INT8 outputs. The iPhone run used the half-to-even input rounding described above, which the July Android run predates. Python, Kotlin, and Swift therefore agreed on the tested vectors. Model outputs may still differ on untested inputs, although input quantization now follows the same rounding rule on both sides.

8 Discussion

8.1 What nominal sample count hides

Official guidance asks for a small representative calibration set [22, 33], and prior PTQ work treats clipping, range estimation, and rounding as the levers [23–25]. This study's sharpest observation is that *nominal sample count said nothing*: the sequential-200 and uniform-200 sets have identical size and differ by 13.4 points of held-out directional recall, while the 200- and 1,000-window uniform sets differ by nothing. What mattered was measured coverage — participants spanned and, through them, the input range. Overlapping sliding windows make this failure easy to produce: the toolchain-default pattern of taking the first N windows concentrates an entire calibration set inside a few hours of one or two sensors.

Once the selection is randomized at all, the refinements evaluated here are exhausted. Stratifying by input-range cells on top of participant balancing — the prespecified primary contrast — moves the endpoint by six hundredths of a point. The histogram and min–max observers agree. The feared resolution trade-off, where covering 39–401 mg/dL coarsens the grid by 60%, costs nothing measurable, consistent with a label that depends on the difference between readings rather than their absolute code. For this model class, calibration design reduces to one rule: do not let contiguous window order pick your calibration data.

The mechanism controls localize the sequential-arm error at the input edge: clipping alone reproduces the loss, the quantized grid without clipping is harmless, and the INT8 interior contributes a quarter point. The class asymmetry follows directly — 61% of `falling_fast` windows contain a reading above the sequential range, since fast falls start high. A model that cannot see how high the trajectory started cannot recognize a fast fall.

8.2 Scope of the accuracy illusion

The sequential-arm INT8 model is a worse clinical-shape classifier with better accuracy (0.570 vs. 0.507): clipping pushes predictions toward `stable`, the class holding three quarters of the weight. Any pipeline that gates a quantized model on raw accuracy would have promoted the broken artifact. Directional (minority-class) recall, or any class-balanced metric, exposes it immediately [7]. This is the practical reading of the study for small imbalanced time-series deployments.

Parity and runtime agreement are necessary but orthogonal checks: the Python, Kotlin, and Swift clients agree bit-exactly on dequantized INT8 outputs for the tested vectors, yet parity says nothing about whether the artifact's input range fits the population. Runtime agreement also does not validate the labels, cohort split, probability calibration, clinical safety, or usefulness to a person with diabetes.

9 Limitations

The results should be read with the following limitations in mind.

1. This is a locked, corrected reanalysis, not a fresh test. The held-out participants were scored once before under the flawed aggregate protocol, and the five-class decision was informed by a historical test run. The protocol, samplers, and endpoint were frozen before the new held-out labels were read, but prior exposure cannot be undone by re-splitting [14].

2. One architecture, one quantizer, and one primary cohort. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12]. The sequential-versus-uniform effect was confirmed on two further cohorts with prespecified paired predictions (Table 11), but all other contrasts remain single-cohort [4], and the null results for range stratification, budget, and observer are statements about this 2,909-parameter CNN with per-tensor affine input quantization, not about PTQ in general. Larger models, other datasets, or other quantizers may behave differently [15, 25].

3. Stride-one windows remain overlapping and dependent [13]; uncertainty therefore comes from participant-cluster bootstrap intervals conditional on the fitted models and calibration sets, with model-seed and draw variation averaged inside each bootstrap draw rather than treated as independent participants.

4. The gap-fill mask removes windows around missing readings, but the retained grid still contains formatter re-gridding between adjacent readings, and interpolation precedes windowing, so retrospective preprocessing may use information unavailable in real time. The three frozen sensitivity tiers (exact-observed windows; gap-filled inputs with gap-free endpoints; any gap-filled endpoint) reproduce both headline findings (Table 10), but the tier evaluation ran after the primary analysis was published, and the gap-filled-endpoint tier's labels score agreement with the formatter's interpolated grid, not with unobserved sensor readings.

5. The five labels are deterministic transformations of a future CGM point, not expert annotations, and the CGM-only model omits meal, insulin, activity, illness, and other covariates. Probability calibration and event-level performance were not evaluated.

6. The nine prespecified artifacts were device-timed on one iPhone (Table 13); repeated calls on one phone are not evidence across devices, the Android device from the historical observations is no longer available, initialization was measured within-process rather than process-cold, and energy and

memory were not measured. The remaining 101 artifacts were not device-timed; they share one operator inventory and byte size with the timed ones.

7. The application replays synthetic or locally extracted historical traces. It does not ingest a live sensor stream and has not undergone prospective, external, usability, or clinical validation.

10 Reproducibility and Ethics

10.1 Code, artifacts, and computational record

The repository contains the training, conversion, benchmark, Android, iOS, and test code. The bundled artifacts were generated from repository commit `361148a` with seed 0. Their SHA-256 digests are `eb96c7e...08e6` (float) and `0dc35387...ea29` (INT8); full hashes are stored beside the assets and asserted by instrumentation tests. Python unit tests cover label boundaries, segment-aware windows, model shape, conversion helpers, synthetic replay behavior, and the provenance audit's split bookkeeping. Android unit tests cover replay, buffering, quantization, latency aggregation, and view-model behavior. A merged- manifest check enforces the absence of Internet permission. The iOS test suite covers the same units and the golden-vector parity. Continuous integration runs it on a simulator, together with the source scan for networking APIs.

The conversion requirements record LiteRT Torch 0.9.1, TorchAO 0.17.0, and Python LiteRT 2.1.5. They also document PyTorch 2.12.1 and NumPy 2.5.0 in the historical environment, although the top-level requirements pinned older versions. These records do not constitute a complete environment lock. PyTorch seed 0 and the formatter's split seed 0 control separate randomization steps; neither guarantees bitwise retraining across platforms. The artifact hashes identify the exact models used for replay.

For the September manuscript revision, we reused the July results and calculated descriptive summaries from the rounded tables. We did not retrain models, rerun evaluation on the research data, or collect new phone timings. Machine-readable run reports and the training checkpoint can be regenerated locally, but are not versioned paper supplements. A fresh run may produce different numbers from those recorded here.

A subsequent provenance audit reran only the GlucoBench formatter and window construction; it involved no training, model evaluation, or held-out glucose values.

The October revision adds the iOS client and its device measurements, recorded on 10 October 2026 with the unchanged artifacts (same SHA-256 hashes). It involved no retraining, conversion, or evaluation on research data. Its aggregate output, including the GlucoBench commit and a SHA-256 hash of the Weinstock data file, is versioned as `paper/results/split_audit.json`.

The calibration-coverage study itself is versioned under `paper/results/calibration-coverage/`: the frozen protocol (`protocol.json`), the sanitized study environment (`environment.txt`; a single virtual environment with the conversion pins, PyTorch 2.12.1, and NumPy 2.5.3, reconciling the previously recorded drift), the eligibility audit (`data_availability.json`), the complete aggregate results with contrasts, intervals, coverage, and all 100 planned cells (`aggregate_r1.json`), and the artifact summary — operator inventories, sizes, and distinct-hash counts (`artifacts_summary.json`); per-artifact SHA-256 hashes live in the local run manifest. Checkpoints, selected window keys, per-window predictions, and participant-level summaries remain local and are regenerable from the committed runner (`experiments/calibration_coverage.py`) with GlucoBench cloned; every published number regenerates from the cached predictions via one analysis command. After the study's publication within the repository, the bundled application artifacts were replaced by its seed-0 float and uniform-200 INT8 models, with hashes and provenance recorded in the application's model manifest.

10.2 Data availability and research-use scope

The CGM data are public research data accessed through GlucoBench under the source dataset's reuse terms [1]. They are not committed to the GlucoEdge repository or the distributed Android build. The committed replay trace is synthetic. This work makes no diagnosis, monitoring, treatment, or dosing claim. Its outputs must not be used for medical decisions. The application displays this restriction in the primary interface.

Code, bundled model files, and synthetic golden vectors are available at `https://github.com/mohamedmenasy/GlucoEdge`; source CGM access is described by GlucoBench [1]. The work uses secondary research data and the prototype has not been studied with newly recruited participants. We do not claim an institutional ethics determination for this work.

10.3 AI assistance and author declarations

OpenAI Codex assisted with language editing during the September manuscript revision, checked sources, and compared claims with repository code and recorded results. Claude Code (Anthropic) wrote the provenance audit script and the Kotlin rounding fix, implemented the iOS client, implemented and executed the calibration-coverage study pipeline under the frozen protocol, and drafted the corresponding manuscript revisions. The draft requires author review. Funding, com-

peting interests, and CRediT author contributions remain to be confirmed by the author before submission.

11 Conclusion

On a corrected, provenance-masked protocol with five training seeds and 100 paired INT8 artifacts, calibration-data coverage fully determined whether static INT8 quantization damaged this compact CGM trend classifier. The toolchain-default sequential selection, whose 200 windows span two participants and cap the input range at 251 mg/dL, cost 13.4 points of held-out directional recall while raising raw accuracy — an illusion a class-balanced metric exposes. Any randomized selection of the same 200 windows restored float-level directional recall; range-aware stratification, a five-fold budget, and a min–max observer moved it by at most eight hundredths of a point; and clip-only controls attributed the loss to input-range clipping rather than grid resolution. Every INT8 artifact is byte-identical in size and selection takes milliseconds, so the corrected calibration is free at deployment time. In this setting, coverage means covering participants; sample count, stratification, and observer choice were the wrong knobs.

The conclusions are scoped to one primary cohort, one architecture, and one quantizer, on a reanalysis of previously touched data; the frozen sensitivity tiers reproduce both headline findings, and the paired-round device benchmark measured the calibration fix as latency-free on the one remaining phone, and the sequential-versus-uniform effect was confirmed on two further cohorts with prespecified paired predictions whose magnitudes track the calibration-coverage deficit. GlucoEdge remains a research prototype, not a medical device.

References

[1] Renat Sergazinov, Elizabeth Chun, Valeriya Rogovchenko, Nathaniel Fernandes, Nicholas Kasman, and Irina Gaynanova. GlucoBench: Curated list of continuous glucose monitoring datasets with prediction benchmarks. In *International Conference on Learning Representations*, 2024. URL `https://openreview.net/forum?id=cUSNs8nGaV`.

[2] Kui Liu, Linyi Li, Yifei Ma, Jun Jiang, Zhenhua Liu, Zichen Ye, Shuang Liu, Chen Pu, Changsheng Chen, and Yi Wan. Machine learning models for blood glucose level prediction in patients with diabetes mellitus: Systematic review and network meta-analysis. *JMIR Medical Informatics*, 11:e47833, 2023. doi: 10.2196/47833.

[3] Andrea Facchinetti, Giovanni Sparacino, Emanuele Trifoglio, and Claudio Cobelli. A new index to optimally design and compare continuous glucose monitoring glucose prediction algorithms. *Diabetes Technology & Therapeutics*, 13(2):111–119, 2011. doi: 10.1089/dia.2010.0151.

[4] Sarala Ghimire, Turgay Celik, Martin Gerdes, and Christian W. Omlin. Deep learning for blood glucose level prediction: How well do models generalize across different data sets? *PLOS ONE*, 19(9):e0310801, 2024. doi: 10.1371/journal.pone.0310801.

[5] Jeremy Pettus and Steven V. Edelman. Recommendations for using real-time continuous glucose monitoring (rtCGM) data for insulin adjustments in type 1 diabetes. *Journal of Diabetes Science and Technology*, 11(1):138–147, 2017. doi: 10.1177/1932296816663747.

[6] Melanie Rodacki, Luis Eduardo Calliari, Antonio Carlos Ramalho, et al. Using trend arrows in continuous glucose monitoring systems for insulin adjustment in clinical practice: Brazilian diabetes society position statement. *Diabetology & Metabolic Syndrome*, 13(1):2, 2021. doi: 10.1186/s13098-020-00607-2.

[7] Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.

[8] Jaques Reifman, Srinivasan Rajaraman, Andrei Gribok, and W. Kenneth Ward. Predictive monitoring for improved management of glucose levels. *Journal of Diabetes Science and Technology*, 1(4):478–486, 2007. doi: 10.1177/193229680700100405.

[9] Sadegh Mirshekarian, Hui Shen, Razvan Bunescu, and Cindy Marling. LSTMs and neural attention models for blood glucose prediction: Comparative experiments on real and synthetic data. In *41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 706–712, 2019. doi: 10.1109/EMBC.2019.8856940.

[10] John Martinsson, Alexander Schliep, Björn Eliasson, and Olof Mogren. Blood glucose prediction with variance estimation using recurrent neural networks. *Journal of Healthcare Informatics Research*, 4(1):1–18, 2020. doi: 10.1007/s41666-019-00059-y.

[11] Kezhi Li, John Daniels, Chengyuan Liu, Pau Herrero, and Pantelis Georgiou. Convolutional recurrent neural networks for glucose prediction. *IEEE Journal of Biomedical and Health Informatics*, 24(2):603–613, 2020. doi: 10.1109/JBHI.2019.2908488.

[12] Ruth S. Weinstock, Stephanie N. DuBose, Richard M. Bergenstal, et al. Risk factors associated with severe hypoglycemia in older adults with type 1 diabetes. *Diabetes Care*, 39(4):603–610, 2016. doi: 10.2337/dc15-1426.

[13] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018. doi: 10.1016/j.csda.2017.11.003.

[14] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.

[15] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021. URL `https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html`.

[16] Serkan Kiranyaz, Onur Avci, Osama Abdeljaber, Turker Ince, Moncef Gabbouj, and Daniel J. Inman. 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151:107398, 2021. doi: 10.1016/j.ymssp.2020.107398.

[17] Nicholas D. Lane, Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro, and Fahim Kawsar. DeepX: A software accelerator for low-power deep learning inference on mobile devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, pages 1–12, 2016. doi: 10.1109/IPSN.2016.7460664.

[18] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. AI benchmark: All about deep learning on smartphones in 2019. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, pages 3617–3635, 2019. doi: 10.1109/ICCVW.2019.00447.

[19] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, et al. MLPerf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html.

[20] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.

[21] PyTorch. PT2E quantization, 2026. URL https://docs.pytorch.org/ao/stable/pt2e_quantization/index.html. Accessed 2026-09-19.

[22] Google AI Edge. Representative dataset for model quantization. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/api/tflite/python/tf/lite/RepresentativeDataset. Accessed 2026-09-19.

[23] Ron Banner, Yury Nahshan, and Daniel Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html.

[24] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7197–7206, 2020. URL https://proceedings.mlr.press/v119/nagel20a.html.

[25] Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. Accurate post training quantization with small calibration sets. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4466–4475, 2021. URL https://proceedings.mlr.press/v139/hubara21a.html.

[26] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

[27] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[28] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://jmlr.org/papers/v12/pedregosa11a.html.

[29] NumPy Developers. numpy.round. NumPy reference documentation, 2026. URL https://numpy.org/doc/stable/reference/generated/numpy.round.html. Accessed 2026-09-19.

[30] JetBrains. kotlin.math.round. Kotlin standard library reference, 2026. URL https://kotlinlang.org/api/core/kotlin-stdlib/kotlin.math/round.html. Accessed 2026-10-09.

[31] Apple. FloatingPointRoundingRule.toNearestOrEven. Swift standard library documentation, 2026. URL https://developer.apple.com/documentation/swift/floatingpointroundingrule/tonearestoreven. Accessed 2026-10-10.

[32] Google AI Edge. Run LiteRT compiled model API on Android with Kotlin. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/next/android_kotlin. Accessed 2026-07-20.

[33] Google AI Edge. Post-training quantization with LiteRT. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization. Accessed 2026-09-19.